

移动应用开发(Android)

实验 I - I

zhjing@scut.edu.cn

教案及提交作业: lms.scutnc.cn

目标

- 1. 创建 Android 开发环境
- 2. 了解并熟悉启动 Activity
- 3. 了解基本的 UI 元素使用

任务

- 1. 创建 Android 开发环境
 - 运行第1个 HelloWorld App (不用写入实验报告)
 - 在模拟器或者真机上运行，确定开发环境安装正常
- 2. 三选一实验：
 - 实现案例 GeoQuiz App，增加音乐播放功能，运行
 - 实现本教案中的“WriteNumberGame”，运行
 - 运行 Android 某官方案例，并分析其结构

1. Android 开发环境

Android 开发环境

developer.android.com

- 1. JDK, Android SDK
 - java —version
- 2. IDE: Android Studio
 - developer.android.google.cn
- 3. 创建并运行 HelloWorld App

Project WriteNumberGame

Project WriteNumberGame

- 步骤 1: 使用 Empty Activity 模板创建 WriteNumberGame 项目
- 步骤 2: 设计启动界面
 - 5秒后, 跳转到主界面

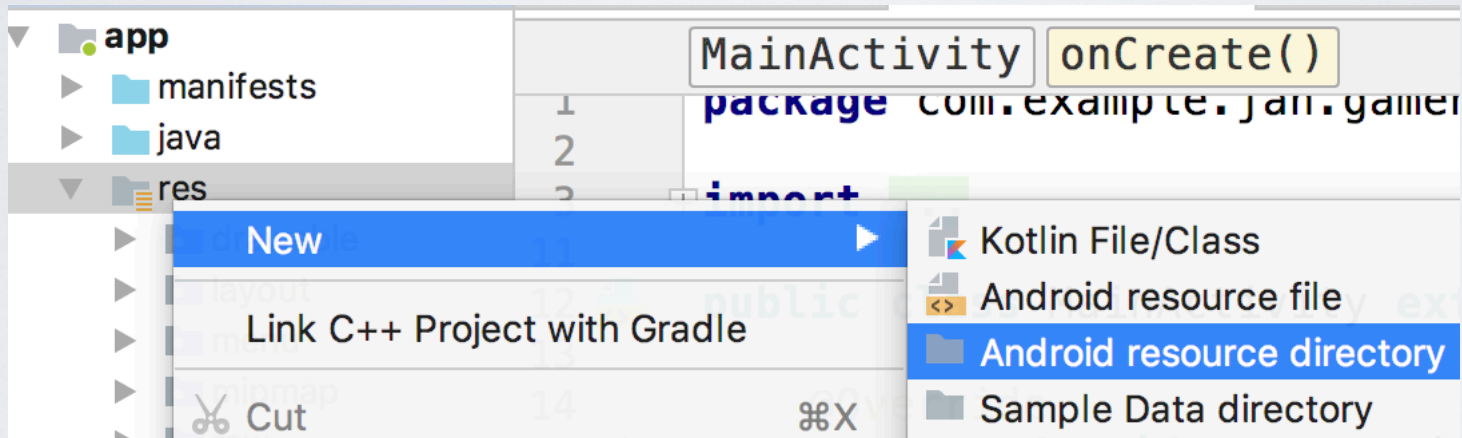


步骤 2: Launch to the Start Activity

- 1. 导入必需资源(resources).
- 2. 修改 layout.
- 3. 使 activity 全屏.

1. 导入资源(resources)

- 如果有较多图片，则创建新目录：
(resource directory)

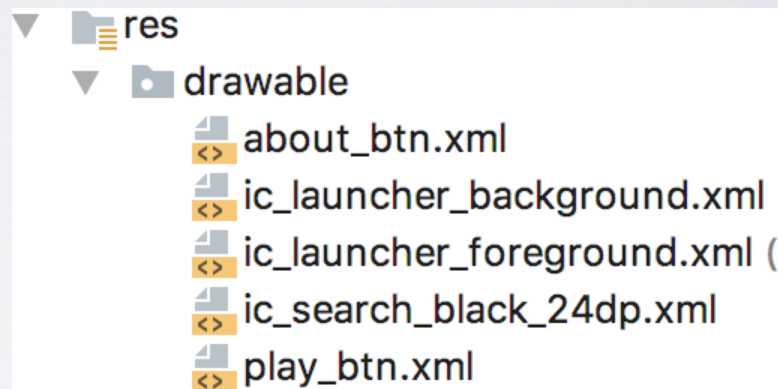


- copy+paste

1. 导入资源(resources)

- drawable 目录里的文件:

- xml
- png
- jpg...



play_btn.xml, 文件中的btn_play*.png 在目录res/mipmap下

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="@mipmap/btn_play1" android:state_focused="false" android:state_pressed="false" />
    <item android:drawable="@mipmap/btn_play2" android:state_focused="false" android:state_pressed="true" />
    <item android:drawable="@mipmap/btn_play2" android:state_focused="false" android:state_pressed="true" />
</selector>
```

2. 修改 layout

- activity_main.xml
 - background etc.
- 在模拟器上运行项目
- LinearLayout

```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@mipmap/start_bg"
    tools:context="com.example.jan.gamenumnumber.MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```

3. 检查 Manifest 文件

```
public class MainActivity extends Activity {
```

```
<?xml version="1.0" encoding="utf-8"?>|
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.jan.gamenum" >

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="GameNumber"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity
            android:name=".MainActivity"
            android:theme="@android:style/Theme.NoTitleBar.Fullscreen">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

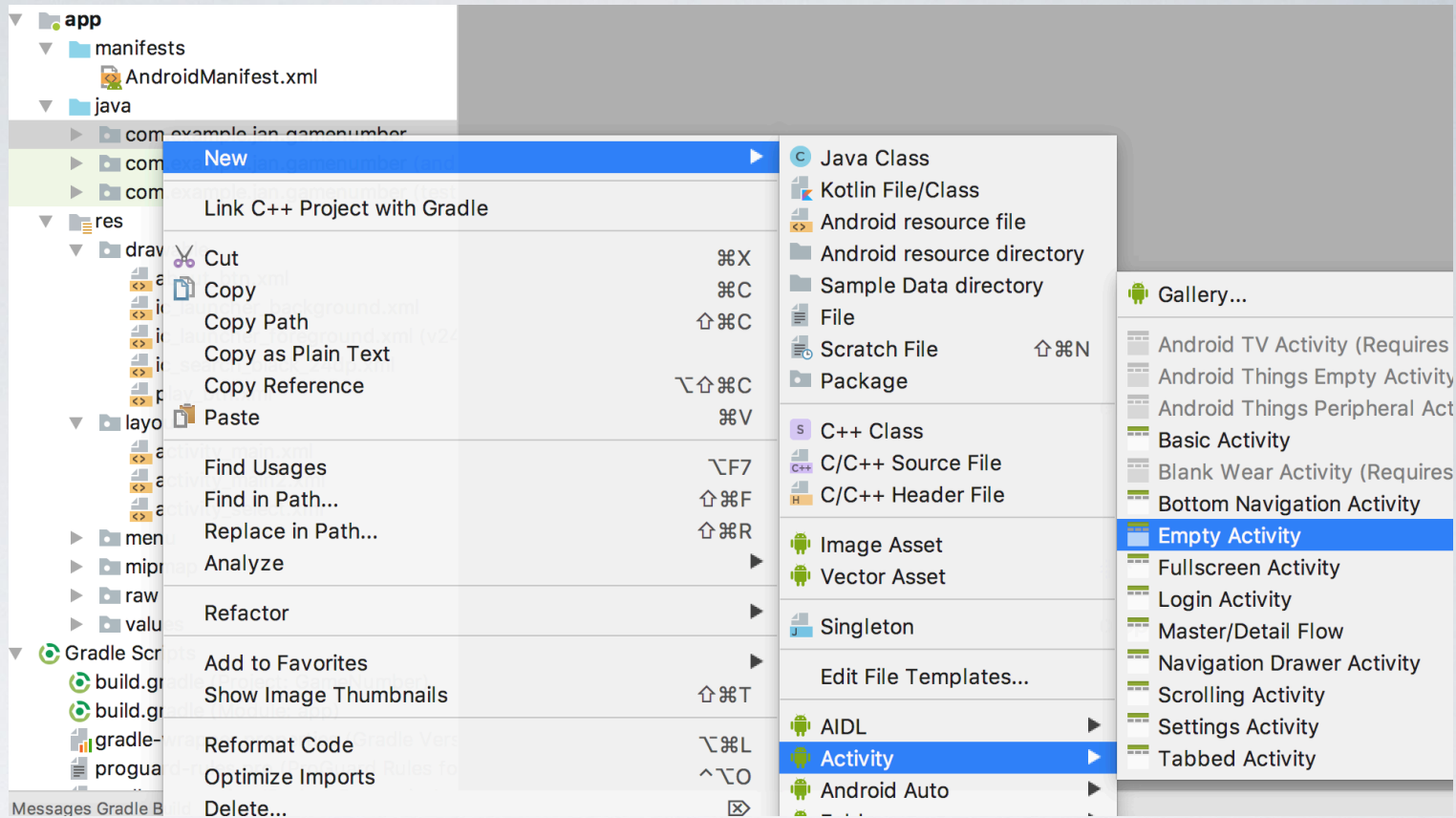
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
```


步骤 4. The Main Game Activity

- 4. 从启动 Activity 跳转到 主 Activity.
- 5. 设计对应的 layout.

4. 从启动 Activity 跳转到 主 Activity

- 刚才启动界面，再创建新的空 Activity 作为游戏的主界面，提供游戏功能.
- 启动界面 5秒后跳转，转变为游戏主界面
- 确保模拟器上运行无误



```
public class MainActivity extends Activity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        Timer timer=new Timer();
```

```
        TimerTask timerTask =new TimerTask() {
```

```
            @Override
```

```
            public void run() {
```

```
                startActivity(new Intent( packageContext: MainActivity.this,Main2Activity.class));
```

```
                finish();
```

```
            }
```

```
        };
```

```
        timer.schedule(timerTask, delay: 5000);
```

```
    }
```

TimerTask

Summary

```
public abstract class TimerTask  
extends Object implements Runnable
```

```
java.lang.Object
```

```
↳ java.util.TimerTask
```

A task that can be scheduled for one-time or repeated execution by a Timer.

Summary

Protected constructors

```
TimerTask\(\)
```

Creates a new timer task.

Public methods

```
boolean
```

```
cancel\(\)
```

Cancels this timer task.

```
abstract  
void
```

```
run\(\)
```

The action to be performed by this timer task.

5. 设计对应的 layout

- 图片放入 drawable or mipmap.
 - `R.drawable.pic1`; `R.mipmap.pic2`
- 创建新的 raw 资源目录，用于保存音乐文件
- 创建用于游戏主界面 MainActivity2 类对应的 layout
 - button-1: (onPlay) 跳转到 Digit-Selection 界面.
 - button-2: (onAbout) 跳转到 About 界面.
 - button-3: (onMusic) 用于 Play or Stop 音乐.

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@mipmap/main_bg"
    tools:context="com.example.jan.gamenumber.Main2Activity">

    <Button
        android:layout_width="90dp"
        android:layout_height="90dp"
        android:layout_above="@+id/btn_music"
        android:layout_centerHorizontal="true"
        android:background="@drawable/play_btn"
        android:onClick="onPlay"/>

    <Button
        android:layout_width="90dp"
        android:layout_height="90dp"
        android:id="@+id/btn_music"
        android:layout_alignParentBottom="true"
        android:layout_margin="10dp"
        android:background="@mipmap/btn_music1"
        android:onClick="onMusic" />

    <Button
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:id="@+id/btn_music2"
        android:layout_alignParentBottom="true"
        android:layout_alignParentRight="true"
        android:layout_margin="10dp"
        android:background="@mipmap/btn_music1"
        android:onClick="onAbout" />
</RelativeLayout>

```

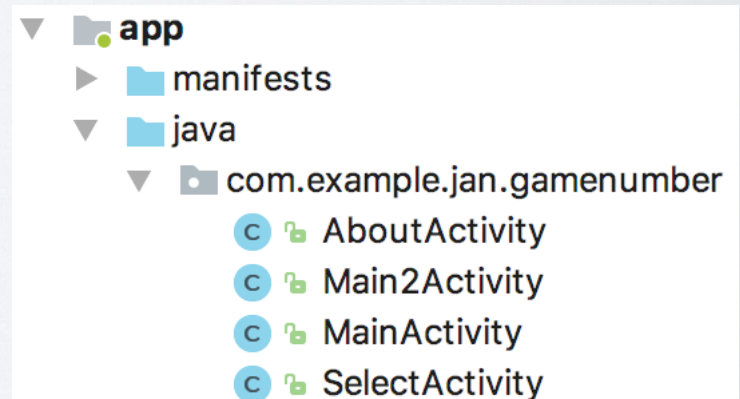


步骤 5. 要启动的其它 Activities

- 7. 创建 Digit-Selection Activity 和 About Activity.
- 8. 设计 Digit-Selection Activity 对应的界面.
- 9. 设计 About Activity 对应的界面.
- 10. 从游戏主界面 Activity 跳转到这2个 Activity.
- 11. 在项目里播放音乐.

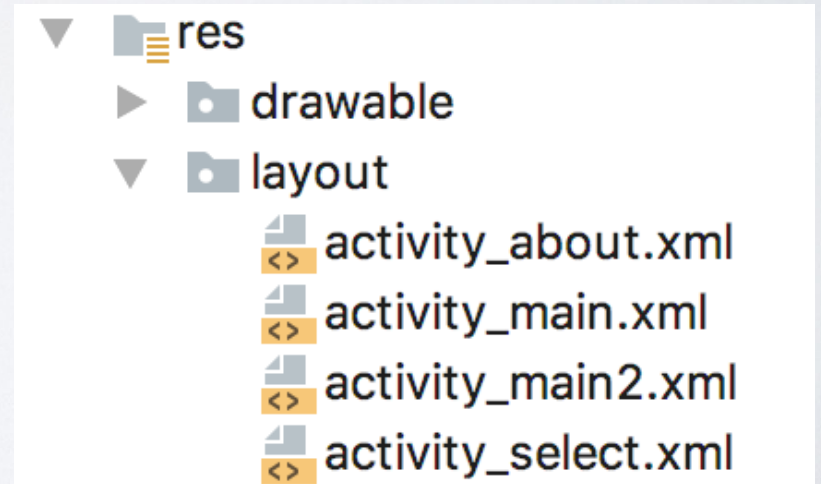
7. 创建 Digit-Selection 和 About Activity

- 用 Empty Activity 模板创建 SelectActivity
- 用 Empty Activity 模板创建 AboutActivity



8/9. 设计对应的界面

- 将图片放入 drawable 或 mipmap 目录
- 将音乐文件 (mp3) 放入 raw 资源目录写 layout:
 - activity_about.xml
 - activity_select.xml
- 加入各种需要的控件



10. 从主界面跳转到2个界面

- SelectActivity: onPlay()
- AboutActivity: onAbout()

```
public class Main2Activity extends AppCompatActivity {  
    ...  
    public void onAbout(View view) {  
        startActivity(new Intent(Main2Activity.this,AboutActivity.class));  
    }  
  
    public void onPlay(View view) {  
        startActivity(new Intent(Main2Activity.this,SelectActivity.class));  
    }  
    ...  
}
```

11. 在项目里播放音乐

- 功能描述：
 1. 用户点击按钮时，播放或者停止音乐
 - onMusic
 2. 当用户启动项目，自动播放音乐。

```

public class Main2Activity extends AppCompatActivity {
    "" ""
    static boolean isPlay=true; //Music playing status
    MediaPlayer mediaPlayer; //Music player object
    Button music_btn; //Music playing button

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main2);
        music_btn=(Button)findViewById(R.id.btn_music);
        PlayMusic();
    }

    public void onMusic(View view) {
        if(isPlay){
            if(mediaPlayer!=null){
                mediaPlayer.stop();
                music_btn.setBackgroundResource(R.mipmap.btn_music2);
                isPlay=false;
            }
        }else {
            PlayMusic();
            music_btn.setBackgroundResource(R.mipmap.btn_music1);
            isPlay=true;
        }
    }
}
"" ""
}

```


11. 在项目里播放音乐

- 3. 当其它 Activity 启动，自动停止播放
- 4. 当主界面出现，重新播放音乐

```
private void PlayMusic() {    //播放背景音乐方法
    //创建音乐播放器对象并加载播放音乐文件
    mediaPlayer = MediaPlayer.create(this, R.raw.start_music);
    mediaPlayer.setLooping(true);    //设置音乐循环播放
    mediaPlayer.start();    //启动播放音乐
}
```

```
public class Main2Activity extends AppCompatActivity {
```

```
... ..
```

```
@Override
```

```
protected void onStop() {
```

```
    super.onStop();
```

```
    if(mediaPlayer!=null){
```

```
        mediaPlayer.stop();
```

```
    }
```

```
}
```

```
@Override
```

```
protected void onDestroy() {
```

```
    super.onDestroy();
```

```
    if(mediaPlayer!=null){
```

```
        mediaPlayer.stop();
```

```
        mediaPlayer.release();
```

```
        mediaPlayer=null;
```

```
    }
```

```
}
```

```
@Override
```

```
protected void onRestart() {
```

```
    super.onRestart();
```

```
    if(isPlay==true){
```

```
        PlayMusic();
```

```
    }
```

```
}
```

```
... ..  
}
```

Activity Lifecycle

MediaPlayer

```
public class MediaPlayer  
extends Object implements VolumeAutomation
```

```
java.lang.Object
```

```
↳ android.media.MediaPlayer
```

added in API level 1

Summary: [Nested Classes](#) | [Constants](#) |
[Fields](#) | [Ctors](#) | [Methods](#) | [Protected](#)
[Methods](#) | [Inherited Methods](#)

MediaPlayer class can be used to control playback of audio/video files and streams. An example on how to use the methods in this class can be found in [VideoView](#).

通过本实验，学习

- 1. 界面跳转功能
- 2. 定时功能
- 3. 音乐播放功能
- 4. 界面之间的数据传递